

Интеграция ИИ в справочник классических танцевальных движений для обучения и контроля исполнения

Варианты использования ИИ в танцевальном справочнике

Интеграция искусственного интеллекта (ИИ) может значительно расширить возможности цифрового справочника классических танцевальных движений. Ниже перечислены основные направления использования ИИ в подобной системе:

- **Распознавание и классификация движений по видео** – ИИ способен идентифицировать, какое танцевальное движение выполняет ученик, на основе видеопотока с камеры. Система с компьютерным зрением может в реальном времени определять позы танцора и сопоставлять их с эталонными движениями из справочника ¹. Это позволит автоматически **определять название движения** (например, battement tendu jeté и др.) и проверять, правильно ли оно выполняется.
- **Автоматическая генерация обратной связи** – На основе данных распознавания поз система может выдавать подробные рекомендации и поправки по технике. ИИ-коуч будет анализировать отклонения от правильной формы и в текстовой форме сообщать ученику, что нужно скорректировать (например: «Поднимите правую руку чуть выше и выпрямите локоть» или «Сильнее вытяните носок левой ноги») ². В современной реализации возможно использование больших языковых моделей для генерации человекопонятных советов на основе выявленных ошибок.
- **Персонализированные рекомендации и план обучения** – Система ИИ может отслеживать прогресс каждого ученика: какие движения уже освоены, где возникают ошибки, какая динамика улучшений. На базе этих данных можно **рекомендовать индивидуальные программы тренировок**, подборку следующих движений для изучения, либо дополнительные упражнения для исправления слабых мест. Например, если у пользователя трудности с выворотностью ноги в battement tendu, система порекомендует упражнения на растяжку или более простые элементы для постепенного улучшения.
- **Адаптивное обучение и сложность** – ИИ может подстраивать уровень сложности и темп обучения под конкретного пользователя. Новичкам будет предлагаться более медленное выполнение с пошаговой разбивкой движений, тогда как продвинутым танцорам – более сложные вариации или **ускоренный темп**. Если система видит, что ученик быстро прогрессирует, она может разблокировать более сложные комбинации; если же возникают ошибки, ИИ замедлит темп или детальнее разберёт проблемные элементы.

- **Интеллектуальный поиск и сопоставление** – Справочник с ИИ позволит искать движения не только по названию, но и по описанию или даже по демонстрации. Пользователь может, например, показать движением или загрузить видео, а система определит, **какое это упражнение**, и покажет информацию о нём. Либо пользователь задаёт вопрос на естественном языке («Как правильно выполнять battement tendu jeté?») – чат-бот на базе ИИ найдет соответствующее описание в справочнике и выдаст разъяснение.
- **Аналитика и отслеживание прогресса** – За счёт сохранения данных о позах и ошибках ИИ-система может строить аналитические отчёты: графики улучшения техники, статистику по углам наклона, амплитуде движений и т.п. Например, можно отслеживать, как уменьшается отклонение угла подъёма ноги в арабеске со временем. Такие **метрики прогресса** помогут мотивировать ученика и объективно оценивать его уровень.
- **Генерация учебного контента** – Алгоритмы генеративного ИИ способны **создавать вспомогательный контент**. В контексте танцев это может означать генерацию анимаций или дополнительных видео с различного ракурса для каждого движения, синтез новых комбинаций на основе изученных, либо автоматическое составление описаний движений. Например, система может с помощью нейронной сети по короткому описанию создать анимированный скелет, выполняющий данное движение, если отсутствует реальное видео.
- **Виртуальный наставник и геймификация** – ИИ может выступать в роли виртуального тренера, который не только даёт поправки, но и поддерживает мотивацию. Можно внедрить **элементы игры**: начисление баллов за правильно выполненные элементы, визуальные эффекты за прогресс (как “фейерверки” на экране за особенно точное повторение ³) и соревновательные челленджи. Такой подход повысит вовлечённость учеников, превращая отработку движений в интерактивную игру с обратной связью в реальном времени.

Таким образом, ИИ в танцевальном приложении может охватить весь цикл обучения – от распознавания и оценки движений до формирования индивидуальной траектории обучения и мотивации учащегося. Далее рассмотрим технические аспекты реализации, особенно для ключевой функции – распознавания поз пользователя через камеру и выдачи обратной связи в реальном времени.

Реализация системы с распознаванием движений в реальном времени

Одной из главных задач является организация **видео-интерфейса**, через который пользователь может выполнять движения перед камерой, а система ИИ будет распознавать позы и сравнивать их с эталонными в режиме реального времени. Для этого требуется решить несколько подзадач: выбор модели позового трекинга, подготовка данных для обучения, алгоритм сравнения с эталоном, визуализация ошибок, архитектура (локально или через сервер) и выбор технологий для веб. Рассмотрим каждый аспект подробно.

Модели отслеживания позы (pose estimation)

Существует несколько популярных моделей и библиотек для **определения ключевых точек тела** на изображении или видео. Они различаются по точности, скорости работы и возможностям. Ниже приведено сравнение наиболее распространенных решений:

Модель	Подход	Ключевые точки	Особенности производительности	Комментарий
OpenPose (CMU)	Bottom-up, Part Affinity Fields ⁴	До 135 точек (включая тело, руки, лицо, ступни) ⁵	Высокая точность позы, поддержка нескольких людей одновременно. Тяжеловесна: на CPU низкая FPS, требует GPU для realtime. Чувствительна к частичным перекрытиям тела (теряет трекинг при сильной окклюзии) ⁶ .	Первое открытое решение реального времени для позы; хорошо подходит для анализа детальной техники (пальцы, стопы), но в веб-контексте сложна.
MediaPipe Pose (Google BlazePose)	Top-down (детектор + трекер) ⁷	33 точки тела (полный скелет с доп. опорными точками) ⁸	Оптимизирована для мобильных устройств и браузера; работает в реальном времени даже на CPU ⁷ . Точность хорошая, хотя чуть уступает современным специализированным моделям. Поддерживает только одного человека в кадре.	Удобна модульная архитектура (можно дополнять определением рук, лица и пр. ⁷). Есть готовые решения в MediaPipe для веб и мобильных приложений.

Модель	Подход	Ключевые точки	Особенности производительности	Комментарий
MoveNet (Google TF Hub)	Bottom-up, одностадийная CNN ⁹	17 точек тела (основные суставы) ⁹	Очень высокая скорость и точность для одного человека ⁶ . Варианты: Lightning (облегчённая, ~50+ FPS на мобильных) и Thunder (усиленная для точности). Малый вес модели. Устойчива к частичному скрытию частей тела ⁶ . Нельзя дообучить (предобученная фиксированная модель) ¹⁰ .	Отличный выбор для веб: есть версия TensorFlow.js, даёт баланс точности и производительности лучше, чем PoseNet ¹¹ ⁶ . Также существует MultiPose вариант (отслеживание нескольких людей).
PoseNet (Google)	Bottom-up (CNN с последующей декодировкой позы)	17 точек тела	Умеренная точность и скорость. Разработана раньше MoveNet и BlazePose, на сегодняшний день уступает им в точности обнаружения поз ¹² . Может работать в браузере через TensorFlow.js.	Может использоваться, если необходима простота и совместимость, но рекомендуется более современный MoveNet как прямой преемник.
Другие (AlphaPose, HRNet, etc.)	Разные (top-down HRNet, alternative OpenPose и др.)	17-136 точек (различные модели)	Многие академические модели обеспечивают высокую точность, но обычно медленнее. HRNet, например, точен, но тяжёл для realtime на CPU. AlphaPose – улучшенная версия OpenPose, тоже требовательна.	Такие модели можно рассматривать для серверной стороны или оффлайн-анализа, если нужно максимум точности на отдельных кадрах, но для online обучения танцам предпочтительны оптимизированные решения выше.

Таблица 1: Сравнение популярных моделей позового трекинга для оценки танцевальных движений.

Как видно из таблицы, для веб-интерфейса **наиболее подходящими являются MediaPipe Pose и MoveNet** – они дают достаточную точность при высоком FPS на обычных устройствах. MediaPipe

привлекательна готовыми модулями и поддержкой на разных платформах, тогда как MoveNet (особенно версия Lightning) показывает лучшую производительность и точность на одном человеке ⁶. OpenPose, хоть и точная, в контексте веб-приложения слишком тяжеловесна и требовательна; её стоит рассматривать только если планируется сервер, выполняющий обработку на GPU, либо если нужна поддержка нескольких танцоров в кадре и детальных точек (например, пальцы для контроля жеста рук).

Веб-реализация позового трекинга: Приоритетным вариантом будет использовать библиотеку *TensorFlow.js* с предобученной моделью MoveNet или PoseNet – это позволит выполнять позу-оценку прямо в браузере, без отправки видео на сервер. Альтернативно, Google предоставляет *MediaPipe JS*, который через WebAssembly выполняет BlazePose на высоких скоростях. Согласно отчётам, MediaPipe на CPU может давать ~15 FPS в браузере, тогда как ранние TensorFlow-based решения давали лишь 2–3 FPS ¹³. Таким образом, современные оптимизированные модели делают возможным **анализ позы локально на устройстве** в реальном времени.

Датасеты и обучение модели сравнения движений

Чтобы сравнивать движения ученика с эталонными, нужна модель или алгоритм, который **понимает различия в позах во времени**. Для её разработки потребуется набор данных (**датасет**), содержащий примеры правильного выполнения каждого движения, а желательно и ошибочных вариантов. Есть несколько подходов к получению таких данных:

- **Использование существующих датасетов по танцам или движениям.** Крупных открытых датасетов именно по классическому балету может не быть, но существуют наборы по общим позам и действиям. Например, датасеты по спортивным упражнениям, йоге или общим действиям (NTU RGB+D и др.) включают позы, которые можно адаптировать. Также в исследовательских работах встречаются наборы данных по танцам: напр., проект AIST++ содержит ключевые точки для современных танцевальных номеров, а некоторые академические статьи формировали собственные коллекции балетных поз ¹⁴. Если удастся найти открытые данные с балетными позициями (возможно, в приложениях к научным статьям ¹⁵), это упростит задачу.
- **Сбор собственного датасета с экспертами.** Практически, наиболее надёжный путь – **записать видео профессиональных исполнителей**, выполняющих базовые движения (battement, jeté, arabesque и т.д.), и создать эталонную библиотеку. Например, команда исследователей из Стэнфорда собрала базу из 32 видео профессионального балета, разрезанных на 145 фрагментов, каждый из которых представляет отдельную «фигуру» (элемент) ¹⁴. Видео можно снимать с фиксированной камеры в стандартных ракурсах (фронтально, сбоку) для каждого движения. Далее, с помощью выбранной модели позового трекинга (например, MoveNet) эти видео преобразуются в последовательности координат скелета, которые и составят эталон.
- **Аннотирование и очистка данных.** Собранные видео/позы важно **аннотировать** – указать, где начинается и заканчивается конкретное движение, какие кадры считать ключевыми фазами. Эталонные последовательности можно очистить: отнормировать позы (например, выравнивать по центру таза, масштабировать под рост) ¹⁶, чтобы сравнение было инвариантным к росту и положению в кадре. Также имеет смысл увеличить вариативность

эталона: если возможно, записать нескольких танцоров, чтобы учесть различия стилей. В упомянутом проекте для каждого движения использовались несколько исполнений: например, 10 ключевых поз хореографии брались из 4 разных видео, чтобы охватить разброс манер ¹⁷ ¹⁸ .

- **Генерация ошибочных примеров.** Чтобы модель научилась распознавать ошибки, в датасет полезно включить **примеры неправильного выполнения**. Их можно получить, попросив учеников разных уровней выполнить движения (и потом привлечь эксперта, чтобы пометить, что сделано неверно), либо сгенерировать искусственно. Например, зная, что правильный угол подъёма ноги = 90°, можно с помощью аугментации данных слегка исказить эталонные позы (уменьшить угол до 70°, добавить наклон корпуса) – такие *синтетические ошибки* помогут алгоритму научиться различать норму и отклонения. Авторы упомянутой балетной системы применяли аугментации к эталонным данным (случайные вращения, добавление шума, вариации скорости) для повышения устойчивости модели ¹⁴ ¹⁹ .

- **Разметка ключевых метрик.** Помимо самих поз, полезно заранее определить, какие метрики важны для оценки качества выполнения. Например, выворотность, высота подъёма ног, выпрямленность колен, положение рук относительно корпуса. Эти доменно-специфичные показатели могут быть вычислены из ключевых точек (углы в суставах, расстояния между точками) и тоже включены в обучающий набор данных, либо использоваться как целевые признаки для выдачи обратной связи.

На основе подготовленного датасета можно обучить модель, которая **сравнивает последовательность поз ученика с эталоном и выдаёт оценку схожести или выявляет ошибки**. Существует два основных подхода к такой модели:

- **Классический алгоритмический подход:** не создавать отдельную нейросеть для оценки, а заложить алгоритм сравнения поз. Например, вычислять среднеквадратичное отклонение координат ключевых точек в каждый момент, разницу углов в суставах между позой ученика и ближайшей по времени эталонной позой, и на основе этого получать метрику соответствия. Если метрика превышает порог – фиксировать ошибку. Такой подход может быть усилен методом *Dynamic Time Warping (DTW)* – динамической нелинейной растяжкой времени для выравнивания последовательностей ²⁰ ²¹ . DTW позволяет соотнести кадры ученика и образца даже если они выполнялись с разной скоростью, находя оптимальное соответствие во времени между двумя последовательностями поз. Алгоритмический подход понятен и прозрачен: разработчик сам решает, насколько отклонение по каждому параметру считается значительным, и может дать интерпретируемую причину (например: угол в колене отличается на 15° от нормы – значит, нога недостаточно выпрямлена).

- **Обучение модели машинного обучения:** используя размеченные данные (правильное/неправильное выполнение, степень отклонения), можно тренировать модель, которая **классифицирует или регрессирует качество движения**. Например, ряд проектов успешно применяли методы классификации по ключевым точкам: запись позы в CSV и обучение модели Random Forest или SVM для классификации “правильно/неправильно” для каждого элемента ²² ²³ . В одном случае для каждого ключевого положения танца обучались отдельные модели (логистическая регрессия, решающее дерево, градиентный бустинг и т.п.) для определения, правильно ли выполнен данный фрагмент ²⁴ . Такой подход позволял

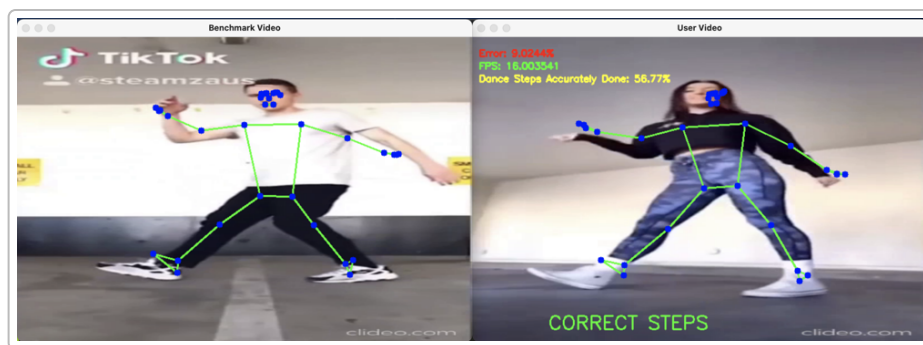
подсветить проблемные места во время проигрывания танца: зелёный цвет – если поза близка к эталону, жёлтый – есть отклонения, красный – поза неверна ²⁵ . Альтернативно, можно обучить глубокую модель: например, нейронную сеть, которая по последовательности поз выдаёт оценку. Современные исследования предлагают использовать **Transformer-модели** для анализа серии движений: так, Vision Transformer успешно применялся для классификации сегментов балетного танца по ключевым точкам ²⁶ , показывая высокую точность (82% на валидации для распознавания элементов) на своем наборе данных. В сочетании с DTW такая модель способна более гибко оценивать соответствие, допуская вариации стиля и порядка движений ²⁷ .

Выбор подхода зависит от ресурсов и необходимой точности. Правилом хорошего тона будет начать с **простого алгоритмического решения** (сравнение углов, DTW) для обеспечения базовой функциональности, и параллельно экспериментировать с ML-моделями на собранных данных для улучшения качества обратной связи. В итоге возможно комбинированное решение: например, использовать ML-модель для общей оценки схожести, а конкретные ошибки детектировать по порогам отдельных метрик (чтобы генерировать осмысленные комментарии вроде «руки слишком широко» или «недостаточный подъём ноги»).

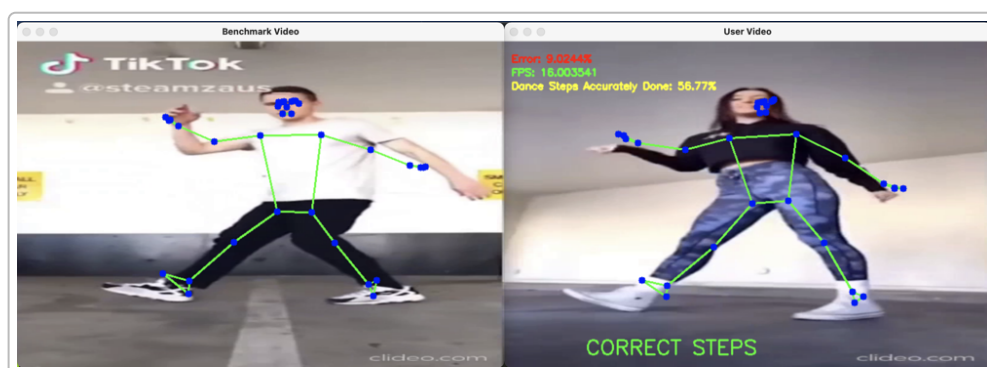
Оценка отклонений от эталона и генерация обратной связи

Метрики отклонения. После того, как у нас есть позы пользователя и эталонная последовательность, необходимо количественно измерить, **насколько точно повторено движение**. Основные метрики могут включать:

- *Разность в углах* ключевых суставов: для каждого кадра (или ключевой фазы) вычисляются углы (например, между бедром и голенью, между корпусом и бедром и т.д.) у эталона и у пользователя. Разница в градусах показывает, насколько ученик отклонился (например, недовыпрямил колено на 10°). Совокупность таких отклонений по всем важным суставам можно свести в интегральный показатель или оставлять отдельно для детализации.
- *Расстояния и пропорции*: сравнение относительных расстояний между парными точками (например, расстояние между руками, между поднятой ногой и корпусом). В балете важны геометрические линии – эти расстояния и соотношения (нормированные на рост танцора) дают понятие о “линии” позиции. Их можно сравнить процентно.
- *Временное выравнивание*: оценивать задержки или рассинхрон. Например, эталонная последовательность может быть длиннее или короче – с помощью DTW или другого метода выясняется соответствие кадров. Затем можно определить, **где именно ученик сбивается по времени**, т.е. какие фазы начаты рано или поздно относительно эталона.
- *Общая метрика схожести*: некоторые реализации вводят интегральный «score» в процентах, показывающий насколько в целом движение выполнено правильно. Например, система может вычислить среднее нормированное расстояние между позами эталона и пользователя за всё движение и перевести его в проценты точности. В одной реализации процент правильности шагов выводился для танца (см. рисунок ниже: ~56.77% accuracy) вместе с величиной суммарной ошибки



. Такой показатель удобен для быстрого понимания прогресса (например, улучшился с 60% до 80% после тренировки).



Пример интерфейса сравнения движений: слева показано эталонное видео с наложенным скелетом (ключевые точки и связи), справа – видео пользователя. Система рассчитывает численную ошибку (Error) и процент точно выполненных движений, выводит FPS обработки. Зеленым цветом выделяются «правильные» части позы (надпись CORRECT STEPS), а красным может помечаться обнаруженная ошибка. Подобное визуальное сопоставление наглядно показывает ученику различия в позах.

Визуализация и UI. Важная часть системы – как пользователь увидит свои ошибки. Несколько подходов могут быть реализованы совместно:

- *Наложение скелетов:* отрисовка поверх видео пользователя скелетной модели, повторяющей его движение, и параллельно – скелета эталона (например, другим цветом или “призраком”). Пользователь визуально сравнивает свою позу с целевой. Если сделать прозрачную наложенную фигуру эталона, танцор может буквально “совместить” себя с ней в зеркале экрана.
- *Цветовые индикаторы:* выделение цветом тех частей тела, которые находятся вне допустимого диапазона. Например, если правая рука слишком низко, вся линия руки на экране подсвечивается красным или жёлтым. В проекте DanceVision предлагалось во время ключевых поз отображать индикатор: зелёный – если поза близка к образцу, жёлтый – немного отклоняется, красный – поза неверна ²⁵. Это позволяет даже без чтения текста понять, на что обратить внимание.

- **Текстовые подсказки:** система генерирует краткие текстовые сообщения (в реальном времени или после выполнения движения) с указанием ошибок. Например: «Правый локоть выше, чем нужно – опустите его чуть ниже уровня плеча», или «Держите спину ровнее, не наклоняйтесь вперёд». Такие подсказки могут быть предопределены правилами (если угол спины $> X$, то “выпрямить спину”) либо формироваться динамически. В одной из систем интерфейс выводил короткие комментарии для каждой фигуры, которые уточняли, что поправить ². Текстовую обратную связь можно показывать рядом с видео или над конкретной частью тела.
- **Графические эффекты и геймификация:** для поддержания мотивации можно визуализировать успехи нестандартно. Например, как описано в проекте **Dance of Fireworks**, точность движений переводилась в эффект фейерверка: чем ближе пользователь к правильной позе, тем ярче и богаче анимация фейерверка на экране ³. Это превращает обучение в своего рода игру – правильное выполнение сразу «вознаграждается» эффектом. Подобные игровые механики (очки, медали, прогресс-бары) можно включить в интерфейс.
- **Сравнение видео «бок о бок»:** отображение эталонного видео/анимации и видео пользователя одновременно для наглядности. Пользователь может видеть, как выглядит профессиональное исполнение и своё собственное, синхронизированное по времени. Такой интерфейс реализован, например, в упомянутом проекте Стэнфорда: UI показывал референс и пользователя рядом, с наложенными скелетами, а внизу краткий текст с замечаниями ²⁸.

В идеале, интерфейс должен сочетать несколько методов: **сразу после выполнения движения** пользователь получает визуальный разбор (видео с наложениями) и текстовый фидбек. Если система работает в реальном времени (например, пользователь танцует перед камерой без остановки), то можно выдавать подсказки на ходу (голосом или текстом на экране) – однако важно, чтобы это не сбивало. Возможно, лучше реализовать режимы: тренировочный (пошаговый, с остановками и подсказками после каждого повтора) и проверочный (танец целиком с итоговым разбором).

Примечание о 3D vs 2D позах: Использование одной камеры даёт координаты позы в 2D (плюс иногда относительная глубина z , как в MediaPipe). Некоторые ошибки трудно уловить в плоскости – например, разворот ноги (эн дэор) или отклонение корпуса вперёд/назад, если камера сбоку. Для улучшения оценки можно либо: - устанавливать камеру под разными углами для разных движений (во многих балетных классах упражнения у станка смотрят сбоку, а центровые – спереди), - либо использовать алгоритмы оценки 3D-позы по видео (есть модели, восстанавливающие 3D-скелет из одного вида). Однако такие модели усложняют систему. Альтернативно, можно ограничиться требованием определённого ракурса для пользователя и подготовить эталон под тот же ракурс, чтобы **сравнение было корректным в 2D**.

Архитектура системы: локально в браузере или клиент-сервер

При разработке веб-приложения с функциональностью распознавания движений есть два основных подхода:

1. Полностью на стороне клиента (в браузере): Здесь все вычисления – от получения видеопотока до вывода обратной связи – выполняются в компьютере/устройстве пользователя с помощью JavaScript и, опционально, встроенных ускорений. Такой вариант стал реалистичным благодаря высокоэффективным моделям вроде MoveNet и технологиям вроде WebGL/WebGPU. Преимущества

очевидны: - *Низкая задержка*: нет необходимости отправлять видео по сети, все происходит мгновенно локально. Это критично для реального времени – пользователь сразу видит результат. - *Конфиденциальность*: видео пользователя не покидает устройство, что важно для частных тренировок. - *Масштабируемость*: нагрузка на сервер минимальна, т.к. вычисления распределены по клиентам.

Реализация этого подхода включает подключение в браузере библиотеки для позы (например, TensorFlow.js с моделью MoveNet SinglePose Lightning – вес модели порядка нескольких мегабайт, загружается один раз). Далее, через API камеры (`getUserMedia`) получаем поток, и в каждом кадре (или с некоторым шагом) запускаем модель позы. Полученные ключевые точки сразу обрабатываются JS-кодом: сравниваются с эталоном (хранящимся тоже на клиенте как заранее подготовленные данные или полученным при старте упражнения). Расчёт отклонений и генерация фидбека тоже идут в браузере. Фактически, **браузер становится мини-студией ИИ**, выполняя и нейросетевые вычисления, и логику приложения.

Ограничения чисто клиентского подхода – возможны трудности на очень слабых устройствах (например, старые смартфоны), где даже оптимизированная модель может давать недостаточный FPS. Однако современные мобильные устройства способны выполнять MoveNet ~30 FPS ⁶. Также, загрузка модели занимает время при старте (несколько секунд), но это не критично. Если же потребуется более тяжёлая модель (например, Transformer для анализа последовательности), её уже сложнее запустить в браузере без сильной оптимизации.

2. Клиент-серверная архитектура: В этом варианте фронтенд (браузер) отвечает только за сбор видео и отображение результатов, а основная обработка (инференс позы, анализ) происходит на сервере (бэкенде). Схема следующая: - Браузер захватывает кадры с камеры и отправляет их на сервер (по WebSocket для непрерывности или AJAX запросами для отдельных кадров). Отправлять можно сырые кадры (JPEG) или уже немного сжатые/преобразованные данные. - Сервер принимает поток и пропускает через модель (например, Python-сервер с библиотекой MediaPipe или OpenPose на GPU). Полученные позы сравнивает с эталоном и формирует обратную связь (числа, текст). - Сервер отправляет обратно результаты в браузер, где они отображаются (рисуются скелеты, выводится текст и пр.).

Плюсы серверного подхода: - *Мощные вычисления*: на сервере можно задействовать GPU или более тяжёлые модели, получить чуть более высокую точность или обрабатывать нескольких пользователей одновременно. - *Единая точка обновления*: улучшения модели сразу отражаются для всех, не надо обновлять код у клиента.

Но минусы довольно серьезные: - *Задержки и требование сети*: даже при оптимальной передаче небольшой задержки не избежать (видео+анализ+ответ). Если интернет-подключение медленное или нестабильное, реального времени не получится. - *Нагрузка на сервер*: при росте числа пользователей сервер должен масштабироваться (видео данные очень тяжёлые). Одновременная обработка видеопотоков – ресурсозатратная задача. - *Вопрос конфиденциальности*: видео ученика пересылается на удаленный сервер, что может требовать защиты данных и согласия пользователя.

В контексте учебного приложения приоритетом является **низкая задержка и приватность**, поэтому рекомендовано по возможности выбрать локальную обработку. Однако можно рассмотреть гибриды: - Например, **компромиссный вариант**: выполнить наиболее тяжёлую часть – **определение позы** –

локально (в браузере), а полученные координаты скелета (это всего ~50 чисел на кадр) отправлять на сервер для более сложного анализа. Координаты весят гораздо меньше видео, передача быстрая, и на сервере можно применять продвинутые алгоритмы (например, сверточную сеть или трансформер для сравнения последовательностей), получая в итоге оценки и текст для отправки обратно. Такой архитектурой достигается баланс: трафик небольшой, приватность частичная (сервер видит только абстрактные позы, а не видео), а серверная часть обеспечивает умный анализ. Впрочем, если анализ тоже реализуем простыми формулами, нет смысла выносить его на сервер – его тоже можно воплотить на клиенте в JS.

Мобильное приложение: Хотя задача ставилась для веб-интерфейса, возможна и реализация в мобильном приложении. Архитектурно мобильное приложение ближе к варианту локальной обработки: можно встроить модели (например, использовать TensorFlow Lite или CoreML для позы прямо на устройстве). Преимущество – доступ к **нейросетевым ускорителям** на смартфонах (GPU/NPUs), которые могут ускорить инференс. Кроме того, мобильное приложение может работать оффлайн. Выбор между веб и мобильным может зависеть от целевой аудитории, но веб (с мобильной версией) часто предпочтительнее для более широкого доступа без установки.

Технологии и инструменты для веб-реализации

При создании прототипа веб-приложения с описанным функционалом потребуются следующие технологические компоненты:

- **Языки программирования:** на клиентской стороне неизбежно используется JavaScript или TypeScript (с учетом современных фреймворков). TypeScript поможет при росте проекта для поддержания типизации. На серверной стороне, если она нужна, подойдут Python (богатые ML-библиотеки) или Node.js (особенно если хочется использовать те же JS-модели, например, через `tfjs-node`).
- **Библиотеки и фреймворки фронтенда:** для интерфейса можно использовать любой популярный фреймворк (React, Vue, Angular) либо чистый HTML/Canvas, поскольку основной нестандартный функционал – это работа с видео. Например, с React можно управлять состоянием отображения (показывать видео, результаты), но критические части (захват камеры и запуск модели) могут быть реализованы безотносительно фреймворка. Визуализацию скелета удобно делать на `<canvas>` поверх видео или с использованием SVG. Для более сложной 3D-графики (если, например, потребуется отображать танцора в виде 3D-модели) можно привлечь библиотеки вроде Three.js, но для 2D позы достаточно Canvas API.
- **WebAssembly и WebGPU:** Если понадобятся производительные вычисления, стоит обратить внимание на WebAssembly. Google MediaPipe предлагает готовые WASM-модули – например, можно подключить скрипт `mediapipe.pose` и вызывать его, получая позы. WASM обеспечивает близкую к native скорость. WebGPU – новая технология (к 2025 году уже широко поддерживается), позволяющая выполнять вычисления на GPU из браузера. TensorFlow.js автоматически может использовать WebGL/WebGPU бэкенды для ускорения моделей. Таким образом, тяжелые операции инференса будут параллеливаться на графическом процессоре пользователя.

- **ML-библиотеки:** ключевая библиотека – **TensorFlow.js** (для работы с нейросетями в браузере). В репозитории TensorFlow.js есть предобученные модели MoveNet и PoseNet, готовые к использованию. Например, модель MoveNet Lightning (single pose) можно загрузить одной строкой из TF Hub и получать позы ~50 FPS на современном устройстве ⁶. Если же планируется собственная модель для оценки движений, её можно либо также обучить в TensorFlow.js (но это ограничено), либо обучить офлайн (скажем, Python/TensorFlow или PyTorch), затем экспортировать в формат TF.js или ONNX. ONNX-модели, кстати, тоже могут загружаться в браузере через ONNX.js runtime.
- **Библиотеки визуализации и UI:** для отрисовки скелета подойдёт обычный Canvas 2D – алгоритмически рисуем линии между точками согласно связи (как на изображении выше). Есть и готовые небольшие утилиты: например, в TensorFlow.js примерах есть код для рисования скелета PoseNet. Если нужно рисовать графики прогресса – библиотека d3.js или Plotly. Для интерфейса реального времени также стоит учесть отображение текста (div поверх видео или canvas текст).
- **Коммуникации клиент-сервер:** если всё же используется сервер, придётся реализовать API. Самый простой вариант – REST API эндпоинты на Flask/FastAPI (для Python) или Express (для Node.js), принимающие изображение (или набор ключевых точек) и возвращающие результаты. Но для непрерывного видео лучше применить WebSocket соединение, по которому браузер будет слать сериализованные данные кадров/поз, а сервер отвечать оперативно. Благо, библиотеки вроде Socket.io (для Node) или WebSocket сервер в Python позволяют это сделать. Кроме того, протокол **WebRTC** мог бы пригодиться для стриминга видео с минимальной задержкой, однако в нашем случае WebRTC может быть избыточен – он полезен для р2р видео звонков, а тут достаточно прямого отправления на сервер.
- **Хранилище данных и бэкенд логика:** сам справочник (описания движений, GIF-анимации, видео-эталон) скорее всего будет храниться на сервере или CDN, а на клиент подгружаться по запросу. Можно использовать обычную базу данных или JSON-хранилище для структуры справочника. Если планируется сохранять историю занятий пользователей, прогресс – понадобятся серверная база данных (SQL/NoSQL) и API для сохранения/получения этих данных. Язык бэкенда Python здесь удобен, т.к. интегрируется с ML, но Node.js тоже может справиться, особенно если логика простая и основной упор на фронтенд.
- **Мобильные технологии (опционально):** для нативной мобильной версии – Android (Kotlin/Java) с TensorFlow Lite или ML Kit (Google ML Kit предоставляет готовую Pose API, кстати, схожую с MediaPipe), iOS (Swift) с CoreML or TFLite. Либо сделать кроссплатформенное приложение, например, React Native или Flutter, и подключить там native-модуль для позы (в RN можно мост к ML Kit сделать). Однако, повторимся, современный мобильный браузер вполне способен выполнять JS-реализацию, так что мобильное приложение не является обязательным.

Подытоживая, для веб-варианта ключевой стек будет: **HTML5 + CSS + JavaScript/TypeScript** на фронте, с использованием **TensorFlow.js** (или MediaPipe) для поз, Canvas/WebGL для визуализации; и *при необходимости* Python (Flask/FastAPI) на сервере с библиотекой типа **MediaPipe** (Google Mediapipe имеет и Python API), или OpenPose (через PyOpenPose), или PyTorch. Однако, приоритет – максимальное выполнение на стороне клиента для снижения задержек и упрощения архитектуры.

Примеры готовых решений и полезные библиотеки

В области распознавания движений и обучающих систем накопилось много проектов и библиотек, на которые можно опереться при разработке:

- **OpenPose** – открытая библиотека C++/Python для детекции позы, разработанная в CMU. Может быть полезна как эталон благодаря высокой точности и поддержке множества точек тела ⁵. Например, исследовательские работы по балету применяли OpenPose для извлечения признаков, а затем классические алгоритмы (SVM, Random Forest) для распознавания балетных поз из этих данных ²⁹. OpenPose также умеет оценивать позы нескольких людей одновременно и распознавать детальные положения пальцев и стоп, что может быть полезно для расширения функционала (например, отслеживание положения рук в порт де бра).
- **Google MediaPipe** – набор готовых решений компьютерного зрения. В контексте танцев полезны компоненты **MediaPipe Pose** (BlazePose) для поз, **MediaPipe Holistic** (комбинированный трекер тела, рук, лица) если нужно отслеживать положение головы или мимику. MediaPipe примечательна тем, что доступна на разных платформах: есть версия для Web (через JavaScript или WASM), для Android/iOS (через ML Kit или SDK), и для Python. Эта библиотека предоставляет ускоренные под капотом модели, разработанные специально для мобильных устройств – её использование может сильно ускорить разработку, так как много задач (напр. сглаживание поз во времени) уже решены внутри. Например, MediaPipe Pose можно буквально вызвать несколькими строчками и получить 33 ключевые точки тела ⁷ ⁸, а дальше сосредоточиться на сравнении с эталоном.
- **MoveNet и TensorFlow.js models** – от Google доступен предобученный MoveNet (Lightning и Thunder) как в TF Lite формате (для мобильных и Python), так и в TF.js формате (для браузера). Также есть старый **PoseNet** в tfjs. Эти модели легко интегрируются: например, TF.js имеет метод `tf.loadGraphModel` для загрузки модели по URL, затем вызов `model.execute(input)` выдает координаты ключевых точек. Использование готовой модели с TF.js избавляет от необходимости держать сервер с TF. Для MoveNet есть и вариант **MultiPose** (отслеживает до 5 человек), если вдруг появится сценарий с несколькими танцорами.
- **Kinect Azure или другие сенсоры:** Вопрос не поднимался напрямую, но существуют аппаратно-программные решения, такие как камера Microsoft Kinect или производные, обеспечивающие сразу 3D позы. Если допустить их использование, то SDK Kinect можно получить данные позы без обучения моделей. Однако, для массового пользователя требовать специальное оборудование нецелесообразно, поэтому лучше опираться на обычные камеры и CV.
- **Примеры проектов:**
 - *AI Dance Coach (Stanford CS231N Project)*: Описанный в отчёте проект ²⁷ ²¹ реализует полный цикл: поза (MoveNet) -> классификация движений (Vision Transformer) -> выравнивание с эталоном (Fast DTW) -> генерация текстовых советов. Он показал улучшение точности учеников ~на 10% благодаря обратной связи ³⁰ ³¹. Ключевые

идеи, которые можно заимствовать: сегментация танца на базовые фигуры, использование трансформеров для понимания последовательности, применение DTW для гибкого сравнения, а также формат UI (двойное видео + текст). Хотя код этого проекта может быть не опубликован, концепции из него очень полезны для вдохновения.

- *DanceVision (GitHub: zin288/DanceVision-AI-Driven-Dance-Proficiency-Assessment)*: любопытный проект, где применена MediaPipe и классические ML-алгоритмы для оценки танца (пример с K-Pop хореографией) ³² ²⁴. Они разбивают танец на 10 ключевых поз, собирают данные из нескольких видео, обучают несколько моделей для классификации правильности позы и затем выдают цветовую индикацию на таймлайне видео ²⁵. Проект выложен на GitHub, можно ознакомиться с их подходом к сбору данных и к визуализации ошибок.
 - *Body Movement Comparison (GitHub: PrashantSaikia/Body-Movement-Comparison-with-Mediapipe)*: демонстрирует приложение на Streamlit, сравнивающее танец пользователя с эталонным видео по ключевым точкам. В нём упор на простое **вычисление ошибки** между позами и визуализацию. Они отмечают, что замена TensorFlow на MediaPipe повысила скорость с ~2 FPS до ~15 FPS ¹³, что подтверждает выбор MediaPipe для реального времени. Также дан пример деплоя через Docker/Kubernetes ³³, что может пригодиться, если делается серверный вариант.
 - *DanceWith (GitHub: bgb10/dance-with)*: эксперимент, использующий OpenPose (оптимизированный через Intel OpenVINO) для сравнения поз нескольких людей, чтобы синхронизировать танец ³⁴ ³⁵. Он подсвечивает несинхронные части тела между танцорами. Это показывает, что OpenPose можно ускорить на CPU с помощью OpenVINO, что может быть вариантом для серверной обработки, если нужен многопользовательский сценарий или более подробные точки (пальцы, стопы).
 - *Educational apps*: Существуют коммерческие или полупрофессиональные приложения для смежных задач – например, приложения для йоги с проверкой поз, или фитнес-тренеры с камерой (варианты “AI personal trainer”). Принципы у них схожие: камера + поза + сравнение с правильной техникой. Например, в научной статье сообщается о приложении, дающем обратную связь по упражнениям (приседания, отжимания) для предотвращения травм ³⁶ – оно использовало OpenPose и простые угловые критерии для оценки, показывая пользователю где отклонения. Эти решения подтверждают, что подход (компьютерное зрение + экспертные правила) работает для обучения движению.
 - *Just Dance и аналогичные игры*: хотя это не учебные приложения, популярные игры вроде **Just Dance** (Ubisoft) или **Dance Central** использовали камеру/Kinect для оценки танцев игрока. Там реализован, по сути, подсчёт схожести движений с заложенной хореографией и выдача очков. Под капотом у ранних версий мог быть не ML, а жёстко заданные допуски, но принцип сравнения поз аналогичен. Эти продукты доказывают осуществимость интерактивного танцевального трекинга, а некоторые моддеры и разработчики сообществ даже выпускают любительские версии с OpenPose для ПК, чтобы заменить Kinect.
- **Алгоритмы сравнения последовательностей**: упомянутая *Dynamic Time Warping (DTW)* – доступна в виде готовых библиотек (например, `fastdtw` на Python). Её применение в танцах показано эффективно ²¹, поэтому стоит иметь её в арсенале. Также, если нужен более продвинутый анализ временных рядов поз, можно обратить внимание на библиотеки для сравнения временных серий (SciPy имеет базовые средства).

- **Библиотеки для оценки качества поз в смежных областях:** В спорте и медицине есть решения для анализа осанки, реабилитации и т.п. Например, библиотека `pyke` (Estimating Kinematics Easily) или специализированные решения в физиотерапии. Они могут содержать готовые функции для вычисления углов, выявления отклонений, которые можно переиспользовать. Также в литературе встречаются подходы типа *репертуар движений* – когда заранее сохраняются эталонные последовательности и алгоритм в реальном времени определяет, какое из них выполняется и насколько точно (например, через k-NN на отрезках поз). Если натолкнуться на такую реализацию, её можно адаптировать под наши балетные движения.

В заключение, интеграция ИИ в справочник танцевальных движений представляет собой междисциплинарную задачу, объединяющую **компьютерное зрение, машинное обучение и веб-разработку**. Технически уже сегодня возможна реализация веб-приложения, где пользователь в любой момент может включить камеру и получить мгновенную оценку своего *battement tendu* или другого движения. Для этого надо грамотно выбрать модель позового трекинга (оптимально – MoveNet или MediaPipe Pose для браузера ^{7 9}), собрать или использовать набор эталонных движений, и реализовать понятную систему оценки отклонений (углы, DTW, возможно ML-классификатор) с дружелюбной визуализацией. Опыт существующих проектов и библиотек значительно ускорит разработку – многие кирпичики (детекция поз, расчет углов, даже примеры UI) уже доступны в открытом виде. Осталось их собрать и адаптировать под специфику классического танца, чтобы получить эффективный обучающий инструмент на благо педагогам и ученикам балета.

^{1 4 5} What is OpenPose? A Guide for Beginners.

<https://blog.roboflow.com/what-is-openpose/>

^{2 14 16 19 21 26 27 28 30 31} cs231n.stanford.edu

https://cs231n.stanford.edu/2025/papers/CS_231N_Final_Project_Latex_Writeup_Update%20-%20Henry%20Jingsong%20Zhou.pdf

³ [2505.02690] Dance of Fireworks: An Interactive Broadcast Gymnastics Training System Based on Pose Estimation

<https://arxiv.org/abs/2505.02690>

^{6 7 8 9 10} are there any difference between Mediapipe and MoveNet? : r/tensorflow

https://www.reddit.com/r/tensorflow/comments/rgc03z/are_there_any_difference_between_mediapipe_and/

¹¹ Comparative Analysis of OpenPose, PoseNet, and MoveNet Models ...

<https://www.semanticscholar.org/paper/Comparative-Analysis-of-OpenPose%2C-PoseNet%2C-and-for-Jo-Kim/b8291771dfcdcc4a6fdb940f0872c0f6120d2c13>

¹² Comparative Analysis of Skeleton-Based Human Pose Estimation

<https://www.mdpi.com/1999-5903/14/12/380>

^{13 33} GitHub - PrashantSaikia/Body-Movement-Comparison-with-Mediapipe

<https://github.com/PrashantSaikia/Body-Movement-Comparison-with-Mediapipe>

¹⁵ Topomorphological approach to automatic posture recognition in ...

<https://www.semanticscholar.org/paper/Topomorphological-approach-to-automatic-posture-in-Saha-Konar/d248f587660921bf4fddba125d508423187888da>

17 18 24 25 32 GitHub - zin288/DanceVision-AI-Driven-Dance-Proficiency-Assessment: Pose estimation and prediction using Mediapipe and various ML models

<https://github.com/zin288/DanceVision-AI-Driven-Dance-Proficiency-Assessment>

20 (PDF) Deep Learning-based Pose Estimation in Providing Feedback for Physical Movement: a Review

https://www.researchgate.net/publication/371389518_Deep_Learning-based_Pose_Estimation_in_Providing_Feedback_for_Physical_Movement_a_Review

22 23 Using MediaPipe to detect the YMCA dance | by Patrick Ryan | Medium

<https://patrick-ryan.medium.com/using-mediapipe-to-detect-the-ymca-dance-1e74296ca307>

29 A feature importance study in ballet pose recognition with openpose

<https://pure.uj.ac.za/en/publications/a-feature-importance-study-in-ballet-pose-recognition-with-openpo>

34 35 GitHub - bgb10/dance-with: Dance-with: Dance with your friends with the right pose!

<https://github.com/bgb10/dance-with>

36 A comprehensive analysis of the machine learning pose estimation ...

<https://pmc.ncbi.nlm.nih.gov/articles/PMC11566680/>